Dynamic VNF Orchestration for UAV-aided Emergency Networks: A Learning and Optimization Control Loop Framework

Chuan Pham*, Kim Khoa Nguyen*,
* Department of Electrical Engineering, École de Technologie Supérieure (ETS),
Université du Québec, H3C1K3, Canada,

***Abstract*—Today, the integration of Unmanned Aerial Vehicle (UAV) and open-source 5G frameworks (e.g., OpenAirInterface, Open5GS) becomes an emerging solution to address many communication challenges in various emergency scenarios. While UAVs can provide a rapid and flexible deployment that traditional base stations cannot achieve, open-source 5G frameworks remove the need for proprietary hardware and automate operations by virtualization in telecom deployments. This research investigates emergency scenarios where the current terrestrial network infrastructure fails to satisfy coverage in some specific areas (e.g., forests or maritime environment). We focus on the impact of multi-path routing strategies in the split O-RAN architecture, aiming to optimize the quality of service (QoS) in uncertain and dynamic coverage environments. To address these challenges, we propose an optimization control loop for SFC-enabled services, which is decomposed into two sub-problems: (1) a multi-agent deep deterministic policy gradient (MADDPG)-based method for dynamic VNF scaling and (2) a Block-Successive-Upper-Bound-Minimization (BSUM)-based algorithm for routing optimization. Leveraging the 5G split architecture, we implement and evaluate our proposed solutions in UAV-assisted emergency networks. Our intensive simulations demonstrate that the proposed approach outperforms existing benchmarks in terms of resource efficiency, service reliability and cost-effectiveness.**

***Index Terms*—Virtual network function (VNF), service function chain (SFC), (Open Radio Access Network) ORAN, Block Successive Upper Bound Minimization (BSUM), Reinforcement Learning, Multi-agent Deep Deterministic Policy Gradient (MADDPG).**

I. Introduction

5G and beyond mobile communication networks are projected to enable flexible deployment while delivering significantly faster speeds, higher bandwidth, lower latency, and greater reliability compared to traditional networks [1]. These advancements offer numerous applications such as ultra-low latency services, real-time surveillance, virtual/augmented reality (AR/VR), autonomous vehicles, and mission-critical services. Such technologies can facilitate many potential solutions for emergency scenarios, such as disasters, and rescue operations [1]. However, in such emergency cases (e.g., forests, regions with damaged infrastructure-based terrestrial networks), the system often fails to operate emergency services. To address this problem, unmanned aerial vehicles (UAVs) equipped with 5G functions have been considered as an emerging solution due to rapid and flexible deployment, adaptive coverage and cost-effectiveness.

Leveraging the split O-RAN architecture [2] in which 5G RAN and 5G core functions can be virtualized and flexibly deployed on computing resources as virtual network functions (VNFs) [2]–[4]. By virtualizing and containerizing network services, a UAV can be equipped with 5G functions to deploy a plug-and-play 5G network in areas where traditional infrastructure is unavailable or insufficient. However, to achieve end-to-end (E2E) latency and data rate constraints, such systems face significant challenges due to resource-constrained environments (e.g., battery, payload, and computing resources).

Recent research has primarily focused on optimizing computing resources and network bandwidth for deploying service function chains (SFCs), but it has not focused on specific scenarios of emergency environments. For instance, prior works have explored resource allocation to minimize latency [5] and VNF placement to reduce queuing delays [6], but the impact of multi-path routing strategies and multi-instance VNF scaling on E2E latency has not been thoroughly investigated. Specifically, the dynamic environment (e.g., node locations, link quality or traffic load) can affect scaling decisions and introduces complexities in routing and resource allocation. Furthermore, the interaction between routing strategies and VNF scaling in UAV-assisted networks remains unexplored, particularly in emergency scenarios where traffic patterns are highly unpredictable.

Regarding the rapid network deployment for emergency scenarios, [7]–[9] focus on implementation test-beds without addressing the network service chain optimization as explored in [5], [6]. As shown in Table I, key factors include infrastructure, mobility, QoS requirements, and scalability. In traditional networks, the infrastructure relies on a fixed 5G/6G RAN and core functions while focusing on balancing latency and cost with predefined plans. In contrast, in emergency scenario with UAV-assisted systems, the network topology becomes dynamic because of UAVs' time-varying positions. Although the UAV-based architecture cannot perform as well as tower-based, it focuses more on reliability and meeting rescue operation requirements in which the RAN and core functions are considered scalable to handle bursty, mission-critical traffic with limited battery and resource constraints. For example, in Fig 1, the system has three areas: A1, A2 and A3. The area A1 has a normal network condition, while in A2 and A3 areas, UAV base stations (UAV-BSs) have to be deployed to support emergency services, such as voice

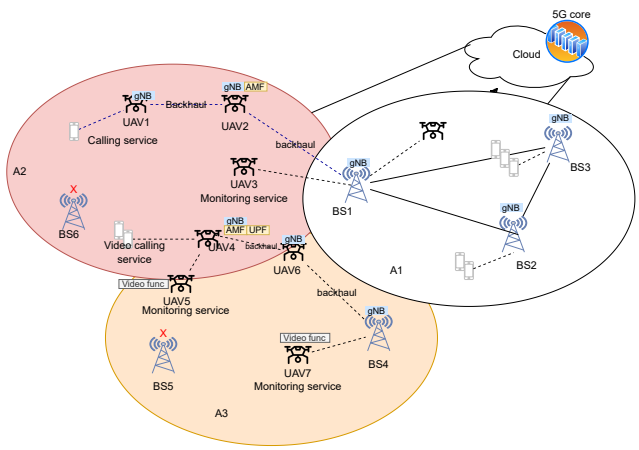

Fig. 1: The system model of VNF-enabled services.

call, video call, and monitoring. Due to the lack of terrestrial towers, some UAVs have to enable backhaul and gNB functions, while others launch 5G core functions (e.g., AMF, UPF) to satisfy the user requests. Due to the UAV's dynamic positions, the system needs to make scaling decisions based on the dynamic topology and the resource constraints (e.g., computing resources, link quality and battery) and optimize routing between VNFs in SFCs to adapt to unpredictable demand.

Therefore, in this work, we study the joint end-to-end (E2E) latency and VNF scaling problem in UAV-assisted emergency networks. Specifically, we consider a UAV-assisted system spanning from access networks to the core, operated by hybrid base stations (e.g., UAV-mounted flying base stations and terrestrial base stations). A set of service function chains (SFCs) must be dynamically scaled to adapt to emergency service demands. Within this system, we implement a control loop where the routing decisions for SFCs are adjusted based on scaling decision, while prior routing configurations influence the subsequent scaling decisions.

To address this, we first formulate a latency-aware optimization problem to minimize the E2E latency through data rate allocation. Then, we address the dynamic VNF scaling problem, based on traffic information from the latency optimization phase to minimize operational costs. These problems are addressed sequentially within the control loop to optimize system performance. Although interdependent, they operate on different time scales: VNF scaling occurs less frequently than routing updates. We leverage machine learning and approximation-based optimization methods to solve these problems, including: i) a Multi-Agent Deep Deterministic Policy Gradient (MADDPG) framework [10], [11] for real-time traffic routing in which each UAV-BS acts as an agent making routing decisions; ii) a Block Successive Upper-Bound Minimization (BSUM) algorithm [12], [13] to decompose the VNF scaling problem into solvable sub-problems.

In summary, our contributions are:

- **Optimized Control Loop for E2E Latency and VNF Scaling**: We propose a framework to jointly optimize traffic routing and VNF scaling for SFCs from access networks to core networks. The framework addresses two interdependent sub-problems:
  1) *Latency-aware minimization*: Optimizes flow routing based on SFC placement to minimize end-to-end (E2E) latency.
  2) *Dynamic VNF scaling*: Minimizes operational costs while adapting to non-stationary traffic demands.
- **MADDPG-BSUM control loop**: We design a control loop that combines: i) a *MADDPG* method for real-time optimization of routing paths and rate allocations; and ii) a *BSUM* algorithm for resource-efficient VNF scaling decisions.
- **Performance evaluation**: Our extensive simulation demonstrates outstanding performance compared to baseline methods in terms of SFC acceptance rates, E2E latency, and low computational overhead.

The rest of our study is organized as follows. In Section II, we discuss selected prior works that relate to our study. In Section III, we present two problem formulations of VNF-enabled service chain deployment for UAV-assisted system including the latency-aware minimization problem **P1** and the dynamic VNF scaling problem **P2**. Section IV discusses the solutions to solve the proposed problems. We then present the simulation result in Section V. Finally, we conclude our work in Section VI.

II. Related Work

In this section, we discuss selected prior works on this topic.

A. Open-source solutions for UAV-assisted networks

Recently, OpenAirInterface (OAI) [14] has been considered as one of the best 5G open-source implementations, following 3GPP standards to provide full software stack for 4G and 5G network. As shown in [15], the authors demonstrated the flexibility of the OAI framework in supporting both LTE and 5G, making it applicable to various emergency use-cases. Similarly, [16], another open-source LTE/5G framework, has a highly active developer community that continuously updates and improve the functionalities and features. However, unlike OAI, which focuses on both 4G/5G RAN and core functions, srsRAN primarily focuses on RAN implementation to ensure high performance and stability.

Open5GS [17] is another open-source framework designed for LTE/5G core network implementations, which supports 3GPP Release 15 and beyond. This framework is lightweight, requiring low resources, making it ideal for UAV-based platforms [18]. While these frameworks have significant potential for different emergency use cases, they require efficient architectures and algorithms for effective management and operation.

B. Applications in emergency scenarios

The integration of UAV into the LTE/5G emergency system has recently gained attentions, focusing on network architectures, resource optimization and deployment strategies. The

Aspect	Traditional telecom scenario	Emergency scenario
Infrastructure	Fixed BS and 5G/6G core	Hybrid (UAV-BS + existing infrastructure)
QoS Requirements	Balanced latency/cost with predefined plans	reliable, low-latency for rescue
Mobility	Static network functions placement with proprietary hardware	Dynamic UAV-BS with time-varying positions and VNF-RAN/Core functions
Scalability	Predictable traffic	Bursty, mission-critical traffic

TABLE I: Comparison of traditional telecom scenario and emergency scenario

authors in [19] highlight the used of UAV-based systems where the ad-hoc modes are emphasized to deal with the unavailability of the terrestrial infrastructure in disasters. In the similar topic, Y.Gao et al. [20] propose a UAV-based architecturei with MIMO antennas to provide reliable communication for emergency cases. This work aligns with [21], which classifies the UAV networks into two paradigms (e.g., centralized and distributed systems).

Focusing on resource allocation in the UAV-based system, Z. Yao et al. [22], A. Ranjan et al. [23] and L. Chiaraviglio et al. [24] are impressive works that have addressed bandwidth assignment and throughput maximization problems with maximizing the user coverage. They validate their 5G-UAV model through real signal propagation models to analyze the performance and identify the specific challenges in the practical implementations.

In the same UAV-assisted topic, [25] models the UAV localization and clustering with swarm intelligence for reducing latency and energy consumption in the self-organizing networks. Similarly, [26] optimizes UAV-based locations for emergency medical services.

Unlike existing work, we focus on a practical UAV-based emergency network where we adopt the 5G open-source platform and the ORAN split architecture to address key challenges, including the efficient placement of 5G functions and the optimization of routing under strict constraints, such as computing resources, link capacity, and battery life.

III. System Model and Problem Statement

A. System model

We begin with a system description to define the scope of our work. We consider an emergency network comprising multi-cell sites covered by existing tower base stations and UAV base stations. The VNF-based network services are placed at multiple nodes (e.g., UAV, edge/cloud nodes). A node has computational resources to deploy different 5G network functions, such as gNB, gateway, 5G core (e.g., AMF, UPF, SMF). The network system is denoted by $G = \{\mathcal{V}, \mathcal{E}\}$, which consists of $|\mathcal{V}|$ nodes, including all the network nodes from the access to the core including UAV-based nodes, base stations, edge nodes, routers and cloud nodes. And, a set $\mathcal{E}$ of physical links connects these network nodes as a graph. We assume that all the nodes are connected to the Internet. Adopting the Integrated Access Backhaul (IAB) mode [27], a UAV can extend the coverage area to another UAV by enabling donor and mobile terminal (MT) functions. All the notations of the system model and problem statement are summarized in Table II.

B. Problem 1: Latency-aware minimization for VNF-enabled services

As shown in Fig. 1, we demonstrate a toy example of an emergency communication network. There is a set of users associated with $|\mathcal{C}|$ SFCs, such as calling, video calling, and monitoring services. These users can be UAVs deployed for emergency operations, victims required assistance, or emergency response workers. We visualize the situation of the system by different colors in which the red and yellow (corresponding to area A1 and A2, respectively) represent the zones with insufficient network coverage. Suppose that each user requests one session. Hence, we assume that there are $|\mathcal{I}|$ client sessions, each requires a minimum flow rate w_i to run its service. We assume that users, who belong to the same service, might have the same minimum rate requirement. A user can connect to different BSs, but at one time slot, we assume that there is only one BS that can serve the UE. Due to the dynamic connectivity between nodes in this system, the network environment (e.g., bandwidth, latency, availability) is changed time by time. We denote the possible paths of flow i as $\mathcal{P}_i = \{P_{i1}, ...P_{ik_i}\}$ and corresponding to each path P_{ij}, we use $L_j, j = 1, ..., k_i$ to present the total latency of path j. For example, the call service can be served by UAV1, UAV2, BS1 and Cloud. Meanwhile the video calling service can be served by UAV4, UAV5, BS4 and Cloud. Note that the UAV is able to operate 5G Core, they can serve UE requests locally without routing to 5G Core in the Cloud. Then, we define the rate allocation variable $w_{ij}(t)$ at time t to allocate for each flow i on each path j.

1) Capacity constraints.: To operate user flows in this system, the total allocated flow data rate should be lower than the bandwidth capacity of each link $e \in \mathcal{E}$. Due to dynamic network conditions (e.g., UAV movement, path loss condition) in the emergency network, for example UAV-to-UAV and UAV-to-ground links, the bandwidth is varying depending on signal quality between them. To formulate this constraint, we denote $a_{e,j}$ as a binary parameter to indicate that link e belongs to path j or not. Hence, we formulate the following constraint:

$$\sum_{i \in \mathcal{I}, j \in \mathcal{P}_i} w_{ij}(t) a_{e,j} \leq B_e, \forall e \in \mathcal{E}, \forall t. \tag{1}$$

2) Minimum Data Rate Guarantee.: We also secure emergency flow i (e.g., victims in coverage areas require a baseline rate for connectivity), by setting the constraint for the data rate allocation as follows:

$$\sum\nolimits_{j \in \mathcal{P}i} wij(t) \geq w_i, \forall i \in \mathcal{I}, \forall t. \tag{2}$$

3) Queuing constraint.: Furthermore, to serve user requests, a node has a processing capacity to queue and process requests. The traffic forwarded to a node could not exceed its capacity. Hence, we model as a backlog queue based on a queuing model and processing capacity of each node v [28]. We denote $b_{v,j}$ as a binary parameter to indicate that node $v \in \mathcal{V}$ belongs to path j or not. Hence, the backlog model constraint is modeled as follows

$$g_v(t) = \sum_{i\in\mathcal{I}, j\in\mathcal{P}_i} w_{ij}(t) b_{v,j}(t) + Q_v(t-1) - \mu_v(t) \leq \epsilon_v, \quad \forall v \in \mathcal{V}, \forall t. \tag{3}$$

where $g_v(t)$ represents the backlog of node v in path j at time t, μ_v is the processing capacity of node v, and ϵ_v is the buffer size of node v. Note that a UAV v can host multiple network function instances depending on its capacity. The larger the buffer size required, the higher the energy consumption.

4) Mission-critical latency constraint.: We formulate a latency constraint for mission-critical services (e.g., UAV-based monitoring service requires a minimum latency). The latency threshold of each service is predefined to guarantee the performance of that service. Therefor, the constraint is:

$$\max_{j\in P_i}\{L_j(t)\phi(w_{ij}(t))\} \leq \bar{L}_i, \forall i \in \mathcal{I}, \tag{4}$$

where $\bar{L}_i$ is the given E2E latency requirement.

5) Objective function.: We take into account the following objective function to minimize the total latency of all critical SFCs by taking into account the worst-case path of session i among all the chosen paths and the penalty term:

$$\mathbb{L}(t) = \sum_{i\in\mathcal{I}} \alpha_i \max_{j\in P_i}\{L_j(t)\phi(w_{ij}(t))\} + \beta \sum_{i\in\mathcal{I}, j=1,\ldots k_i} \phi(w_{ij}(t)) \tag{5}$$

In this optimization problem, we use the binary function $\phi(u) = 1$ if $u \geq 0$, otherwise $\phi(u) = 0$. α_i and β are the weighted parameters belonging to $(0, 1]$. Since this formulation allows multi-path decision, we add the penalty term to mitigate the splitting flow which proportionally increases the packet loss in (6). Therefore, the latency-aware minimization problem for dynamic disaster environments is formulated as follows:

$$\mathbf{P1}: \min_{\mathbf{w},\mathbf{x}} \quad \sum\nolimits_{t=1}^{T} \mathbb{L}(t) \tag{6}$$

$$\text{s.t.} \quad (1) - (4) \tag{7}$$

Due to the dynamic mobility and battery-based operation of UAVs, the processing and network latency of SFCs are affected. Specifically, a UAV might be overloaded when receiving too many requests. As a result, several user sessions will be dropped and the flying time of UAVs is shortened. To address this, a zero-touch orchestration is required to automatically scale the number network functions instances. It needs an entire network observation to have efficient manner. Hence, we next model the dynamic network function scaling problem in the emergency environment to minimize the operational cost incurred in the system.

C. Problem 2: Dynamic VNF scaling optimization problem

We consider a set $\mathcal{C}$ of emergency service chains that are active over time T. To satisfy user demand under UAV energy constraints, we propose to dynamically scale the number of 5G instances and adjust their placement across UAVs. Specifically, we scale up the number of network instances to increase the network performance and reduce the latency and scale down them when the demand drops. We also adjust the location of these instances to minimize network latency and reduce the operational costs incurred by the system. For example, when video call occurs between emergency response workers, the UAV-based system can enable gNB and 5G core functions to process the service locally without routing them to the Cloud 5G Core. However, it will require more energy consumption on UAVs to operate these functions. Therefore, we design a scaling optimization problem for emergency service chain with two primary allocation variables as follows:

i) $x^c_{nv}(t)$, an integer variable, to indicate the number of instances of the nth VNF of the service chain c placed on node v at time t.
ii) $r^c_{n\to(vv')}(t)$, a real variable, to indicate the traffic rate allocation of the nth virtual link of the service chain c over the physical link $(vv') \in \mathcal{E}$.

By controlling these variables, we can make a horizontal and vertical scaling on the UAV-based emergency system. Next, we construct the following constraints and objective function.

1) Service chain constraints: UAV-based nodes have limited computing resources due to payload and battery constraints. We formulate the resource capacity constraint of node v as follows:

$$\sum_{c\in\mathcal{C}} \sum\nolimits_{n\in\mathcal{N}^c} x^c_{nv}(t) p_n \leq \kappa_v p_v, \forall v \in \mathcal{V}, \tag{8}$$

where $\kappa_v \in (0, 1]$ is the decay factor of each node v and p_n is the required resources of an instance of the nth VNF. With the emergency context, we can prioritize redundancy for node v by adjusting κ_v, which can be predefined based on UAV failure logs in practical environments.

2) Minimum VNF instance constraint: To guarantee the resiliency of some critical emergency services, VNF n in the service chain c requires a minimum $I_n(t)$ instances calculated based on urgency.

$$\sum\nolimits_{v\in\mathcal{V}} x^c_{nv}(t) \geq I^c_n(t), \forall n \in \mathcal{N}^c, \forall c \in \mathcal{C}. \tag{9}$$

The minimum number of VNF instance $I^c_n(t)$ can be given by the network operator or calculated based on the data size d_i and the amount of CPU cycle required to process 1-bit of data input at VNF n.

3) Traffic demand constraint for critical SFCs: Given the routing decision in **P1**, at time t the cumulative data rate at the first VNF of SFC c is calculated by

$$\sum_{i\in\mathcal{I}} \sum_{j\in\mathcal{P}_i} w^c_{ij}(t) = \lambda^c(t), \tag{10}$$

where $w^c_{ij}(t)$ is the allocation data rate of SFC c of user i on path j. Thus, to meet minimum throughput for mission-critical

data, we formulate the following constraint

$$\sum\nolimits_{(v,v')\in\mathcal{E}} \alpha_n r^c_{n\to(vv')}(t) \geq \varrho^c \lambda^c(t), \forall n, \forall c, \tag{11}$$

where $\varrho^c \in (0,1]$ is considered as survival threshold for critical SFC. For example, some rescue UAVs can set $\varrho^c = 0.9$ for a monitoring SFC, which means 90% of their data must be satisfied.

4) Link capacity constraint: In addition, the total traffic rate allocated for SFCs should be lower or equal the link capacity. Hence, we have

$$\sum_{c\in\mathcal{C}, n\in\mathcal{N}^c} r^c_{n\to(vv')}(t) \leq B_{(vv')}, \forall v, v', \forall t. \tag{12}$$

5) Latency constraint for emergency services: To account for strict threshold of emergency SFCs, we model the processing queuing latency at nodes and the propagation latency on physical links. Specifically, we consider the processor sharing M/GI/1 queue [29] at each physical node to model the processing queuing delay. This queue happens because of sharing computing resource between multiple VNF instances in the same node with the available processing rate μ_v (depending on UAV payload and battery capacity). Thus, we have the first processing delay constraint at node v as follows:

$$L^{pro}_v(t) = \frac{1}{\mu_v - \sum_{c\in\mathcal{C}, n\in\mathcal{N}^c} \mu_n x^c_{nv}(t)}, \forall v \in \mathcal{V} \tag{13}$$

We obtain the processing latency of the nth VNF by considering the maximum processing latency of its instances as follows:

$$L^{pro,c}_n(t) = \max_{v\in\mathcal{V}} \{L^{pro,c}_v(t) | x^c_{nv}(t) > 0\}, \forall c, \forall n. \tag{14}$$

Given the latency $L_{v,v'}$ between physical nodes v and v' in the network topology, we have the end-to-end (E2E) latency constraint of the service chain c as follows

$$\sum_{n\in\mathcal{N}^c} L^{pro,c}_n(t) + \sum_{n\in\mathcal{N}^c,(v,v')\in\mathcal{E}} L_{(v,v')}\phi(r^c_{n\to(vv')}(t)) \leq \overline{L}_c, \tag{15}$$

where $\sum_{n\in\mathcal{N}^c,(v,v')\in\mathcal{E}} L_{(v,v')}\phi(r^c_{n\to(vv')}(t))$ is the aggregated propagation latency in SFC c and $\overline{L}_c$ is the given E2E latency threshold, which is related to $\bar{L}_i$ in P1.

6) Cost optimization for emergency SFCs: The cost model of SFCs can be defined based on the number of instances deployed in physical nodes over time T. Additionally, scaling VNFs often involves costly operations such as VNF placement and configuration. Therefore, we aim to minimize the number of scaling events by introducing a penalty function in the objective function. Thus, we model the objective function as follows:

$$E(t) = \sum_{c\in\mathcal{C}, n\in\mathcal{N}^c, v\in\mathcal{V}} (\delta_{nv} x^c_{nv}(t) + \sigma_n [x^c_{nv}(t) - x^c_{nv}(t-1)]^+), \tag{16}$$

where δ_{nv} is cost of an instance of the nth VNF at node v (e.g., weighted by energy consumption on UAVs), and σ_n is the operational unit cost to change an instance of the nth VNF (penalty for scaling events).

	Parameters of Network Topology
V	The set of network nodes (UAVs, BSs, edge nodes).
$\mathcal{E}$	The set of physical links.
$\mathcal{I}$	The set of users.
$\mathcal{C}$	The set of emergency SFCs.
$\mathcal{N}$	The set of VNFs.
	Parameters and variables of P1
$\mathcal{P}_i$	The set of paths to route request i.
P_{ij}	The path j to route request i.
B_e	The bandwidth capacity of link e.
w_{ij}	The rate allocation of request i on path j.
g_v	The backlog of node v.
ϵ_n	The buffer size of VNF n.
w_{ij}	The rate allocation variable on path j for user i.
w_i	The minimum rate requirement for the service of user i.
L_j	The total latency on path j.
α_i, β	The weighted parameters.
	Parameters and variables of P2
p_n	The requirement resources of VNF n.
p_v	The resource capacity (e.g., CPU, memory) of node v.
κ_v	The decay factor of node v.
λ_c	The arrival rate of service c.
I^c_n	The number of instances required for function n in SFC c.
ϱ^c	The fractional parameter of requests.
μ_v	The processing capacity of node v.
μ_n	The processing capacity of VNF n.
$\overline{L}_c$	The E2E latency requirement of SFC c.
L	The total latency on path j.
$x^c_{nv}(t)$	The number of instances of the nth VNF of the service chain c placed on node v at time t.
$r^c_{n\to(vv')}(t)$	The allocation rate of virtual link n at time t.
	Functions
$\phi(.)$	The binary function.
$L^{pro}_v(.)$	The processing delay function of node v.
$L^{pro,c}_n(.)$	The processing delay function of node n.
$E(.)$	The cost function.

TABLE II: NOTATION.

Based on the formulated constraints and objective function, we formulate the dynamic VNF scaling problem for emergency SFCs as follows:

$$\begin{aligned}
\textbf{P2}: \min_{\boldsymbol{x},\boldsymbol{r}} \quad & \sum_{t=1}^{T} E(t) \\
\text{s.t.} \quad & (8)-(15), \\
& (x^c_{nv}(t))[1 - M\sum\nolimits_{v'\in\mathcal{V}} r^c_{n\to(v,v')}(t)] \leq 0, \\
& \forall n, \forall c, \forall v, t = 1, ..., T, && (17) \\
& r^c_{n\to(vv')}(t) \geq 0, t = 1, ..., T, \forall n, \forall c, \forall v, v'. && (18)
\end{aligned}$$

We add constraint (17) to ensure that if an instance of the nth VNF is placed on node v, the total traffic at node v should not be negative. We use a large constant M to linearize this constraint as presented in (17).

In general, **P2** is formulated as a Mixed Integer Non Linear Programming (MINLP), which is not trivial to solve **P2** due to the high combination of nodes and links mapping. Furthermore, the coupling constraints (15) and (17) pose significant challenges for state-of-the-art solvers. As presented in **P1** and **P2**, although these problems are independent, they have strong interdependencies, which requires us to address in the next section.

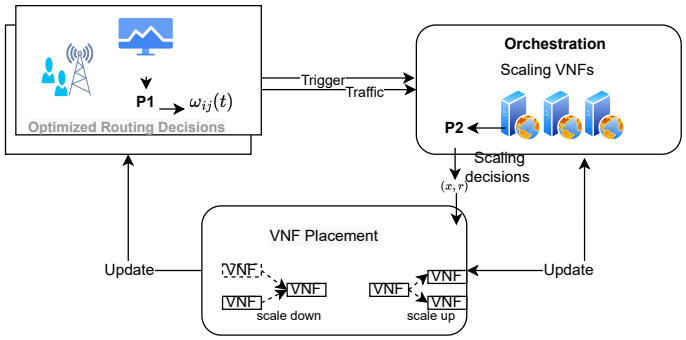

Fig. 2: Control loop diagram for zero-touch emergency SFC management and scaling framework.

IV. Proposed Zero-touch Emergency SFC Management and Scaling Framework

To implement a solution for the proposed optimization model, we design a **zero-touch SFC management system** for emergency communication networks. As illustrated in Fig. 2, our framework adopts a hybrid-distributed centralized architecture.

1) Distributed routing decision module: Each UAV-BS or edge node is equipped with an optimized routing decision module capable of making routing decisions independently. This module automatically operates an algorithm to solve the sub-problem **P1**.
2) Centralized orchestration module: Additionally, there is a centralized module that observes and orchestrate the system. It can monitor the arrival traffic to trigger a VNF scaling event, ensuring that demand traffic is satisfied and resource utilization is optimized during time T. For example, if the number of incoming requests violates the routing constraints of **P1**, the orchestration module will trigger a scaling procedure. Since this module requires complete network information and scaling process takes time to to execute, it should be in the control center or the Cloud.

The system operates as a control loop, shown in Fig. 2. It consists of two main algorithms. In the first phase (routing), the algorithm solves **P1** to route emergency SFC traffic through optimal paths. In the second phase (scaling), if the traffic violates capacity threshold, the centralized module is triggered to optimize resource allocation by solving **P2**. These two phases are interdependent. The scaling results obtained from **P2** will impact the routing decisions made in **P1**, while traffic patterns resulting from **P1** inform future scaling needs.

In the next subsection, to implement this control loop, we propose algorithms based on MADDPG and BSUM frameworks that find near-optimal solutions of **P1** and **P2**.

A. *MADDPG-based algorithm for **P1***

1) Markov game model: In order to solve **P1** in an online fashion, we advocate a multi-agent reinforcement learning framework in this section. We first model the system as a Markov decision process (MDP).

The multi-agent RL problem is defined as a Markov game [10], in which multiple agents interact with the environment via a Markov decision process, similar to the basic RL model. We transform problem **P1** into a partially observable Markov game for U agents corresponding to U base stations in the system, where each agent decides how to route its user requests. A tuple $(U, \mathcal{S}, \mathcal{O}, \mathcal{A}, R, P)$ is used to model the MDP.

- $\mathcal{U} = \{1, 2, ..., U\}$ is the set of agents.
- $\mathcal{S}$ is a set of states. $s \in \mathcal{S}$ consists of the global information of the network, such as link conditions and traffic distributions. Then, according to **P1**, the environment state at time t is written by

$$s(t) = \Big\{\{w_{ij}(t), L_j(t)\}_{i\in\mathcal{I}, j\in\mathcal{J}}, \{B_e(t)\}_{e\in\mathcal{E}}, \{g_v(t)\}_{v\in\mathcal{P}_i, \forall i\in\mathcal{I}}\Big\}, \quad (19)$$

  where the state $s(t)$ is comprised of the data rate routing of flows on each path j, current latency $L_j(t)$, available bandwidth $B_e(t)$ and the backlog $g_v(t)$ of node v.
- $\mathcal{O} = \{\mathcal{O}_u\}_{u\in\mathcal{U}}$ is a joint observation space of all agent, where $o_u \in \mathcal{O}_u$ is the private observation of agent u in its observation space $\mathcal{O}_u$. We describe the observation as follows

$$o_u(t) = \Big\{\{w_{ij}(t), L_j(t)\}_{i\in\mathcal{I}_u, j\in\mathcal{J}_u}, \{B_e(t)\}_{e\in\mathcal{J}_u}, \{Q_n(t)\}_{n\in\mathcal{N}}\Big\}, \quad (20)$$

  where $\mathcal{I}^u$ is the subset of users connected to UAV-BS u and $\mathcal{J}_u$ is the subset of paths going from UAV-BS u. Different from the typical RL model, state s might not be fully accessible by an agent. Instead, a base station has a local observation o_u to make decisions at each time slot.
- $\mathcal{A}$ is a set of joint actions of the agents. Hence, an action is represented as $a = (a_1, a_2, ..., a_U)$. In our work, each action a_u corresponds to a routing decision, given by $a_u(t) = \{w_{ij}(t)\}_{i\in\mathcal{I}_u, j\in\mathcal{J}_u}$.
- $R = \{r_u\}_{u\mathcal{U}}$, where $r_u(s, a, s')_{s,s'\in\mathcal{S}, a\in\mathcal{A}_u}$ is the reward function associated with state s and action a, measuring the effect of the action taken by agent u at a given state s. To optimize the system based on problem **P1**, the reward function is calculated as follows

$$r_u(t) = \sum_{i\in\mathcal{I}_u} \alpha_i \log \frac{L_{ij}(t)\phi(w_{ij}(t))}{\max_{j\in P_i} L_{ij}(t)} + \beta \log \frac{\phi(w_{ij}(t)}{|P_i|}. \quad (21)$$

  In (21), each agent minimizes the reward for its local subset $\mathcal{I}_u$ based on the objective function of **P1**. However, we use a logarithm function to reshape the rewards to avoid large fluctuations during training. Hence, the total reward is as follows

$$r(t) = \frac{1}{U} \sum_{u\in\mathcal{U}} r_u(t). \quad (22)$$

- P is the state transition from s to s' with probability $P(s'|s, a)$.

State s may not be completely visible to agent u in a multi-agent context. As a result, the Q function is based on the private observation o_u as $Q_u(o_u, o_{-u}, a_u, a_{-u})$, where o_{-u} and a_{-u} are the joint observation and action of all agents except for agent u, respectively. The standard Q-learning update calculation is typically simplified to its stateless version as follows

$$Q_u(o_u, o_{-u}, a_u, a_{-u}) = Q_u(o_u, o_{-u}, a_u, a_{-u}) + \gamma[r(t) - Q_u(o_u, o_{-u}, a_u, a_{-u})] \quad (23)$$

2) MADDPG algorithm: Basically, MADDPG is an extended version of the DDPG method [10], [11]. The DDPG algorithm combines the advantages of the gradient policy method and DQN. An agent is made up of two essential parts: an actor and a critic. According to the policy π_u, an action decision is made by the actor for each observation o_u. Then, the critic uses a state-action function $Q_u(.)$ to evaluate the action chosen by the actor. In this work, we employ a combination of centralized and distributed learning models where each UAV-BS (agent) receives an adjustment from the centralized network model with global-coordinated knowledge to achieve better performance. The global critic model uses a global network state s and joint actions of all agents to calculate $Q(s, a)$, which evaluates the action decision of each agent.

Similar to the actor-critic model [10], the actor component makes action decisions according to each observation. On the other hand, the critic component evaluates the action selected by the actor based on the function $Q_u(s, a)$. Consider U UAV-based agents with policies parameterized by $\theta = \{\theta_1, ..., \theta_U\}$ and the set of policies corresponding to each agent, $\pi = \{\pi_1, ...\pi_U\}$, based on [11] the expected return gradient of each agent u is calculated as follows:

$$\nabla_{\theta_u} J(\theta_u) = E_{s\sim p^{\mu}, a_u\sim\pi_u}\left[\nabla_{\theta_u} \log \pi_u(a_u|o_u) Q_u^{\pi}(s, a_1, ..., a_U)\right], \quad (24)$$

where $Q_u^{\pi}(.)$ is the centralized action-value function with the actions of all agents $(a_1, ..., a_U)$ and the global state $s = (o_1, ...o_U)$. For sake of presentation, we use μ_u to denote the policy μ_{θ_u}. Hence, following [11], the gradient can be presented as

$$\nabla_{\theta_u} J(\mu_u) = E_{s,a\sim D, a_u\sim\mu_u(a_u|o_u)}\left[\nabla_{a_u} Q_u^{\mu}(s, a_1, ..., a_U) |_{a_u=\mu_u(o_i)}\right], \quad (25)$$

where D is the relay buffer recording the batch transition of all agents. The centralized Q_u^{μ} is updated by minimizing the loss

$$L(\theta_u) = E[_{s,a,r,s'}(Q_u^{\mu}(s, a_1, ..., a_U) - y)^2],$$

with the target that is calculated by

$$y = r_u + \gamma Q_u^{\mu'}(s', a'_1, ..., a'_U)|a'_u = \mu'_u(o_u))$$

where $\mu' = \{\mu'_{\theta_u}\}_{u=1,...,U}$ is the set of target policies.

We follow [10] to implement the experimental setup, in which all agents' transition data is randomly sampled in a mini-batch D from the experience replay to train the policy, and the target network is introduced as a copy of the Q-function to increase learning stability. The MADDPG-based algorithm for **P1** is presented in Algorithm 1.

Algorithm 1: MADDPG-based Algorithm for **P1**.

Initialization: Randomly initialize environments with U base stations;
Initialize the relay buffer D;
for $episode = 1, ..., Max_Episode$ **do**
  $s \rightarrow$ Reset network environment;
  for $i = 1, ..., |D|$ **do**
    Each BS u selects routing action based on the current policy and exploration $a_u = \mu_{\theta_u}(o_u) + \mathcal{N}_t$;
    BS executes the action to get the new observation o'_u and reward r_u;
    Store transition (s, a, r, s') to the buffer D;
    for $u = 1, ..., |U|$ **do**
      Sample a minibatch of S sample (s^j, a^j, r^j, s'^j) from D;
      Set
      $$y^j = r_u^j + \gamma Q_u^{\mu'}(s'^j, a'_1, ..., a'_U)|a'_u = \mu'_u(o_u^j));$$
      Update critic by
      $$\min L(\theta_u) = \frac{1}{S}\sum_j (y^j - Q_u^{\mu}(s^j, a_1, ..., a_U))^2;$$
      Update actor by
      $$\nabla_{\theta_u} J \approx \frac{1}{S}\sum_{j=1}^{S} \nabla_{\theta_u}\mu_u(o_u^j)\nabla_{a_u} Q_u^{\mu}(s^j, a_1^j, ..., a_U^j)|_{a_u=\mu_u(o_u^j)}; \quad (26)$$
    end
    Update the target network parameters of all agents by $\theta'_u \rightarrow \tau\theta_u + (1-\tau)\theta'_u$;
  end
end

B. Dynamic VNF scaling algorithm

In this sub-section, we continue with the second problem in our predefined system, **P2**, where the traffic information of the entire network is monitored to trigger a scaling decision for SFC c. However, this problem is not straightforward to solve due to the vast number of possible combinations [30].

To address this problem, we first relax the time index t by considering the window time $T = 2$ where the decision at time t that only depends on the previous placement at time $t-1$.In fact, **P2** is still challenging to solve due to the size of decision variables depending on the huge number of combinations of SFCs, VNFs, nodes, and links in the system. To tackle this complexity, we advocate the Block Successive Upper Bound Minimization (BSUM) algorithm, which can decompose the original complicate problem into multiple sub-blocks. BSUM

is well-suited for non-convex and non-smooth optimization problems by leveraging an iterative parallel algorithm.

1) Background of the BSUM framework: Literally, BSUM is a variant of the well-known technique, block coordinate descent (BCD) [31], in which a single block of variables is solved at each iteration while the remaining blocks are left unchanged. Unfortunately, BCD has significant drawbacks when applied to non-convex problems as it may struggle to guarantee convergence. To address these concerns, the authors in [12] introduced BSUM method, offering a more robust and efficient algorithm, which has the following standard form.

$$\min_{\boldsymbol{x}} f(x_i)_{i\in\mathcal{I}},\ \text{s.t.} x_i \in \mathcal{Z}_i, \mathcal{Z} = \mathcal{Z}_1 \times ...\mathcal{Z}_I, \tag{27}$$

where $f(.)$ is the continuous function and $I = |\mathcal{I}|$. For $\forall i \in \mathcal{I}$ and the closed convex set $\mathcal{Z}_i$, we consider x_i as the block of variables. Following the steps of the BCD algorithm, at each iteration k, we solve the following problem to find a solution of a block of variables:

$$x_i^k \in \arg\min f(x_i, \boldsymbol{x}_{-i}^{(k-1)}), \tag{28}$$

where

$$\boldsymbol{x}_{-i}^{(k-1)} = (x^{(t-1)_1,...,x_{i-1}^{(t-1)}}, x_{i+1}^{(t-1)}, ..., x_I^{(t-1)}).$$

Nevertheless, the above sub-problem is still difficult to solve and obtain the convergence in case of non-convex functions. Therefore, the BSUM framework introduces the following proximal upper-bound, constructed by adding quadratic penalization to the objective function:

$$\widetilde{f}(x_i, \widetilde{x}) = f(x_i, \widetilde{x}_{-i}) + \varphi/2(x_i - \widetilde{x}_i), \tag{29}$$

where φ is a positive penalty parameter. Hence, instead solving (28), at each iteration k, we solve the proximal upper-bound function by

$$\begin{cases} x_i^{(k)} \in \arg\min_{x_i\in\mathcal{Z}_i} \widetilde{f}(x_i), \forall i \in \mathcal{I}, \\ x_j^{(k)} := x_j^{(k-1)}, j \neq i. \end{cases} \tag{30}$$

In the state of the art, there are some extensions of the BSUM method that focus on analyzing how to select coordinate j, which has some impacts on the convergence performance [12], [13]. In this problem, we reuse the index i and j in this section to indicate the index of variables.

*2) BSUM-based algorithm for **P2**:* To solve **P2**, we begin by reformulating this problem with relaxing allocation variable $\boldsymbol{x}$, which becomes a continuous form. Hence, the problem **P2** is as follows:

$$\min_{\boldsymbol{x}\in\mathcal{X}, \boldsymbol{r}\in\mathcal{R}} \mathcal{E}(\boldsymbol{x}, \boldsymbol{r}) \tag{31}$$

where

$$\mathcal{E}_{\boldsymbol{x}\in\mathcal{X}, \boldsymbol{r}\in\mathcal{R}} = \sum_{c\in\mathcal{C}, n\in\mathcal{N}^c, v\in\mathcal{V}} (\delta_n x_{nv}^c(t) + \sigma_n[x_{nv}^c(t) - x_{nv}^c(t-1)]^+), \tag{32}$$

Algorithm 2: BSUM-based optimization algorithm for dynamic VNF scaling problem

Input: Traffic: $\boldsymbol{\lambda}^c$, Instances: $\mathbf{I}$, Available resource settings: $\boldsymbol{p}, \boldsymbol{B}, \boldsymbol{\mu}$
Output: Instance vector $\boldsymbol{x}^*$, Rate allocation $\boldsymbol{r}^*$

Initialize: $k = 0$;
Find a feasible starting point $(\boldsymbol{x}^{(0)}, \boldsymbol{r}^{(0)})$;
repeat
  Select index set $\mathcal{I}^k$;
  Solve for instance allocation:
  $$x^{(k+1)} \in \min_{x_i} \mathcal{E}_i(x_i, \boldsymbol{x}^{(k)}, \boldsymbol{r}^{(k)})$$
  Set $x_j^{(k+1)} = x_j^{(k)}$, $\forall j \notin \mathcal{I}^k$;
  Solve (35) to update rate allocation $\boldsymbol{r}^{(k+1)}$;
  Increment $k := k + 1$;
until $||\mathcal{E}^{(k)} - \mathcal{E}^{(k+1)}|| < \epsilon$;
Apply rounding on $\boldsymbol{x}^{(k+1)}$ to ensure integer values;
Return $(\boldsymbol{x}^{(k+1)}, \boldsymbol{r}^{(k+1)})$;

and the feasible set of $\boldsymbol{x}, \boldsymbol{r}$ given by

$$\begin{aligned} \mathcal{X} \triangleq \{\boldsymbol{x} : \sum_{c\in\mathcal{C}} \sum\nolimits_{n\in\mathcal{N}^c} x_{nv}^c(t) p_n \leq \kappa_v p_v, \forall v \in \mathcal{V}, \\ \sum\nolimits_{v\in\mathcal{V}} x_{nv}^c(t) \geq I_n^c(t), \forall n \in \mathcal{N}^c, \forall c \in \mathcal{C}, \\ (x_{nv}^c(t))[1 - M \sum\nolimits_{v\in\mathcal{V}} r_{n\to(v,v')}^c(t)] \leq 0, \forall n, \forall v, v'\} \end{aligned}$$

and

$$\begin{aligned} \mathcal{R} \triangleq \{ \sum\nolimits_{(v,v')\in\mathcal{E}} \alpha_n r_{n\to(vv')}^c(t) \geq \varrho\lambda^c(t), \forall n, \forall c, \forall v, v', \\ \sum_{n\in\mathcal{N}^c} L_n^{pro,c}(t) + \sum_{n\in\mathcal{N}^c, (u,v)\in\mathcal{E}} L_{(u,v)} \phi(r_{n\to(vv')}^c(t)) \leq \overline{L}_c, \} \end{aligned}$$

We next decompose the problem into blocks according to each variable x_i and r_i. Then, the algorithm iteratively minimizes an upper bound of the objective function to sequentially update $\boldsymbol{x}$ and $\boldsymbol{r}$. To achieve this, at each iteration k, a surrogate function is introduced by adding the quadratic term into the objective function, as shown below.

$$\widetilde{\mathcal{E}} := \mathcal{E}(\boldsymbol{x}_i; \boldsymbol{x}^{(k)}, \boldsymbol{r}^{(k)}) + \varphi/2(\boldsymbol{x}_i - \widetilde{\boldsymbol{x}})^2 \tag{33}$$

where φ is a positive parameter. Given the initial feasible points $\widetilde{\boldsymbol{x}}, \widetilde{\boldsymbol{r}}$, we find the minimum solution of (33) by decomposing into blocks. At iteration $k+1$, we solve (34) and (35) to get the optimal solution of (33).

$$\boldsymbol{x}_i^{(k+1)} \in \min_{\boldsymbol{x}_i\in\mathcal{X}} \widetilde{\mathcal{E}}_i(\boldsymbol{x}_i; \boldsymbol{x}^{(k)}, \boldsymbol{r}^{(k)}), \tag{34}$$

$$\boldsymbol{r}_i^{(k+1)} \in \min_{\boldsymbol{r}_i\in\mathcal{R}} \widetilde{\mathcal{E}}_i(\boldsymbol{r}_i; \boldsymbol{x}^{(k+1)}, \boldsymbol{r}^{(k)}). \tag{35}$$

In this algorithm, we adopt the simple cyclic rule for index selection based on [12], where coordinates are chosen in a repeating sequences, i.e., $1, 2, 3, ..., I, 1, 2, 3, ...$ A complete algorithm for dynamic VNF scaling problem is outlined in Alg. 2.

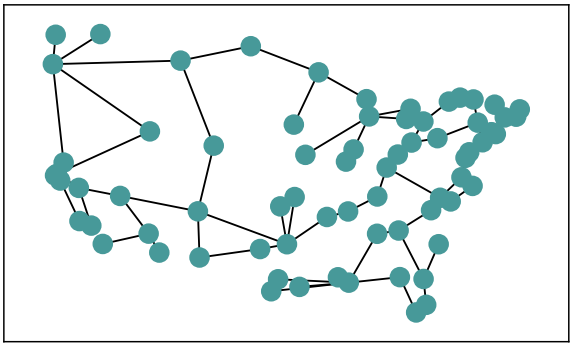

Fig. 3: Network topology.

V. Numerical results

In this section, we evaluate the proposed framework using Python-based simulations for dynamic emergency communication scenarios to assess the performance based on numerical results.

A. Network settings

We adopt the Coronet network topology [32], [33] but modify with hybrid link setting including wire and wireless links to connect 72 nodes. Of these, we select 44 nodes designed as UAV/terrestrial BSs as shown in Fig. 3, while the rest are VNF-enabled nodes (e.g., edge nodes, portable edge servers or UAV-based nodes) that can have computing resources ranging from 32 to 128 cores running each core at 1 GHz to place multiple VNFs of SFCs. Link capacity between physical nodes is created in a range of [1, 10] Gpbs and link latency is set from 5 to 20 ms.

To simulate the real emergency traffic, we randomly generate user requests during 1000 time slots, as shown in Fig. 4, with a normalized workload of 50 SFCs. We categorize them into three types of periods in the system - low, normal, and high workload - highlighted using different colors. Each emergency SFC is chained by 3 to 10 VNFs generated. For example, to cover a 5G service in an emergency area, the control plan SFC in includes a chain of 5 functions $gNB \rightarrow AMF \rightarrow SMF \rightarrow PCF \rightarrow UDM$, while the user plane SFC is comprised of $gNB \rightarrow UPF \rightarrow DN$. The computing requirement of a VNF is set in the range of $[1, 8]$ cores.

We assume that each user has one request belonging to only one SFC. Users are randomly assigned to UAV-BSs at each time slot. A connection between base stations and users is implemented by setting a constant for the path loss factor of 3.3, and additive Gaussian noise power of -174 dBm/Hz, transmission power of 27.0 dBm, and channel bandwidth of 30 MHz [34]. The lifetime of a user request is simulated in a range of [0.5, 10]s, and E2E latency is set between [10, 200]ms for mission critical services. The user request data size and the required CPU cycle of each request are generated between [0.05, 0.1] MB and [5, 20] cycles, respectively. The backlog buffer size is set in a range of [16, 32] MB. The simulation settings are also summarized in Table II.

Description	Values
Network settings	
Number of VNF-enabled nodes	38
Number of BSs	44
Computing resources of nodes	[32, 128] cores
Bandwidth of physical links	[1, 10]Gbps
Latency of physical links	[5, 20]ms
CPU frequency	1GHz
Channel bandwidth	30MHz
Power transmission	27.0 dBm
Path loss factor	3.3
SFC settings	
Number VNFs of an SFC	[3, 10]
Number of SFCs	50
Computing request of a VNF	[1, 8]cores
Backlog buffer	[16, 32]MB
User request settings	
Life time	[10, 30]s
Data size	[0.5, 10]MB
E2E latency	[10, 200]ms
VNF instance price	[0.1, 0.2]

TABLE III: Network settings

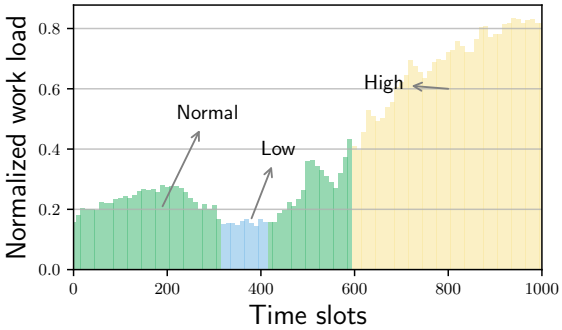

Fig. 4: Request arrivals.

B. Results

1) Convergence evaluation: In this part, we illustrate the convergence results of our proposed methods. First, we evaluate the training performance of MADDPG in addressing **P1**, as shown by the snapshot of average rewards for three BSs in Fig. 5. In the training phase, all the parameters of UAV-BS agents are initialized using TensorFlow. The rewards exhibit fluctuations during the first 500 episodes of MADDPG before gradually reaching the convergence by the adjustment of actors and critics.

Second, we evaluate the convergence of Alg. 2 based on the gap between consecutive iterations, $||\mathcal{E}^{(k)} - \mathcal{E}^{(k+1)}||$. Using the cyclic strategy in BSUM results in a better convergence compared to the random strategy. Although the random approach reaches the convergence faster with less than 100 iterations, the gap is larger than that observed with the cyclic method, as shown in Fig. 6.

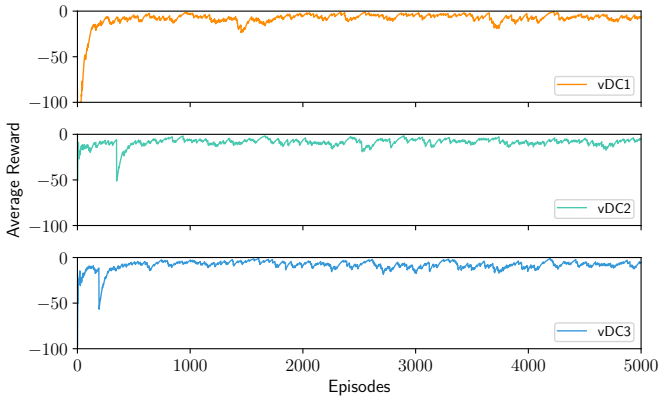

Fig. 5: Convergence of MADDPG-based algorithm for **P1**.

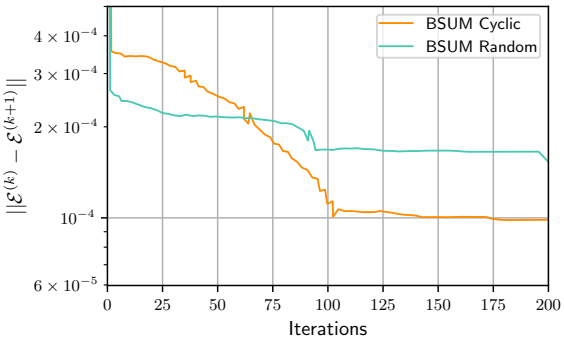

Fig. 6: Convergence of BSUM algorithm for **P2**.

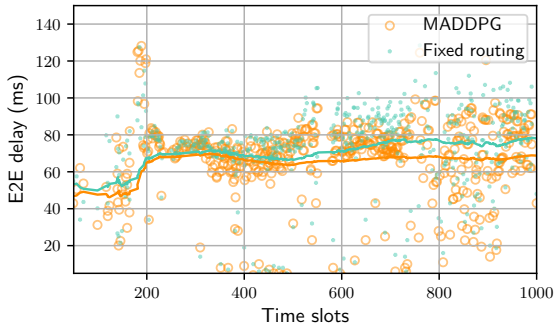

Fig. 7: E2E latency evaluation.

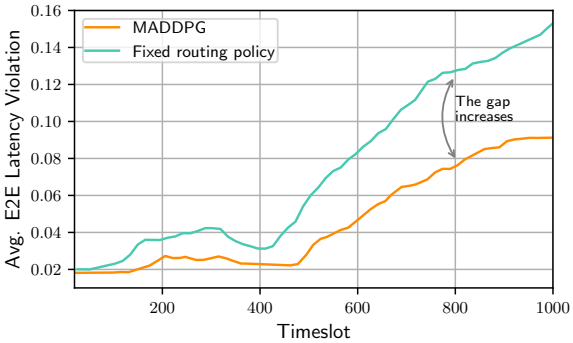

Fig. 8: E2E latency violation.

2) Evaluation of satisfaction: In order to evaluate user satisfaction, we sampled 534 user sessions during 1000 time slots. We compare the average delay for sessions between MADDPG and a fixed routing policy (i.e., using the shortest path), as shown in Fig. 7. From the figure, it illustrates that the fixed routing policy has a similar average latency to MADDPG method at the early stages (i.e., less than 200 time slots, when the system has sufficient resources to accept all the requests.). However, the delay increases more significantly after that, as most of the shortest paths are fully utilized. In addition, the lines in Fig. 7 present the mean delay of all sessions during our simulation period, which further demonstrates the out-performance of MADDPG in controlling the routing.

Fig. 8 compares the latency violation rate between MADDPG and the baseline. In the baseline, user requests from a base station are forwarded equally to SFCs based on the given possible paths from a base station to the first VNF of an SFC, without considering path latency. In this simulation, we gradually increase the workload to evaluate the performance of our proposed method. On average, MADDPG reduces the violation rate by more than 4% compared to the baseline. As the workload increases, the gap between MADDPG and the baseline becomes more significant, further demonstrating the effectiveness of the MADDPG algorithm, espically after 600 time slots.

3) Evaluation of resource allocation: We present the total VNF-instance cost of **P2**, comparing BSUM with the Best-fit baseline, as shown in Fig. 9. To implement the baseline, we randomly set the number of instances at time t within a range between the minimum number of instances $I_n^c(t)$ (e.q. (9)) and the maximum number of instances placed in the past (i.e., $\max_{i=0,\ldots,t-1} I_n^c(i)$). Since this value varies depending on the user requirement, we set randomly in this range to avoid over-provisioning and frequent scale up/down of instances. When the volume of user requests is low (i.e., fewer than 500 time slots), the total costs of both approaches show a small difference. However, BSUM obtains consistently a lower outcome. Additionally, BSUM benefits the system significantly as the number of request is high (after 500 time slots), as observed in Fig. 9. On average, BSUM reduces the number of instances used in the system by more than 2.7%.

In addition, Fig. 9 also presents the resource utilization for both approaches. Note that we calculate the average utilization of active links. Unlike the comparison of the number of instances, the link utilization of BSUM is significantly different from that of Best-fit. The difference arises because Best-fit does not account for the latency and greedily places VNF instances at the best available nodes. Hence, active links of Best-fit are highly utilized. During the simulation period, on average, scaling VNF using the BSUM-based method utilizes 37.9% of active link bandwidth, where Best-fit occupies 43.6%.

4) Evaluation of linear and learning bound parameter.: Finally, we evaluate the impact of VNF scaling based on a setting of the bound parameter I_n^c. Typically, this parameter is calculated using a linear function based on the amount of

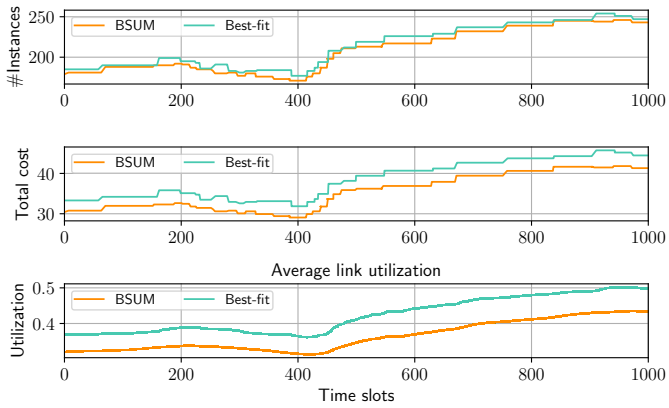

Fig. 9: Evaluation of the resource allocation and system cost.

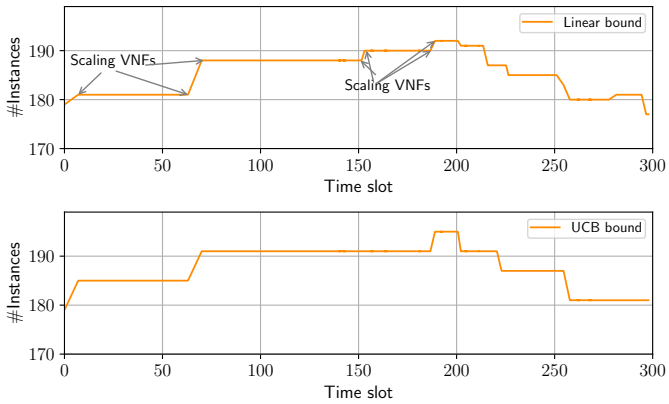

Fig. 10: Impact of the bound parameter on VNF scaling.

workload [35]. However, this method often results in under-allocation when the low workload is low and over-allocation when workload is high [36]. An alternative method is to use Upper Confidence Bound (UCB) approach, which leverages learning-based techniques to determine the bound parameter [37].

Therefore, we apply these settings in our work to assess the impact on the scaling performance. As shown in Fig. 10, the number of VNFs are adjusted at each scaling operation. In the control loop, the bound setting I_n^c influences the resource capacity available to serve the requests. An appropriate setting determines whether a scaling trigger will happen in the next time slot. In the first sub-figure, the setting of I_n^c is based on a linear function. It shows that the number of scaling times is higher than the second setting using UCB. The number of VNF instances reflects the workload variations over 300 time slots, demonstrating effective resource allocation strategy. The second setting, based on UCB, dynamically adjust the number of instances of emergency SFCs. This method reduces the frequency of scaling operations, which implies better stability. However, compared to the first setting method, the number of VNF instances is higher which results in a higher resource cost. These results reveal an interesting trade-off in the scaling problem. Due to the limitation of this study, we do not investigate deeply this issue and assume that the network operators can flexibly choose their settings based on their specific requirements to deploy the emergency communication systems.

VI. Conclusion

This research investigated the challenges of dynamic VNF scaling in UAV-aided emergency networks, explicitly incorporating E2E latency and resource constraints for mission-critical SFCs. Unlike prior studies mainly considered the static SFC placement, we studied a closed-control-loop control framework that jointly optimizes the real-time user session routing and cost-efficient orchestration in the dynamic UAV-based emergency environment. Such a control loop is modeled by **P1** and **P2** corresponding to two interdependent problems: i) latency-aware routing optimization for emergency traffic (**P1**), ii) and dynamic VNF scaling optimization (P2). We leverage MADDPG to solve **P1**, enabling a distributed UAV-based decision solution. For **P2**, we employ BSUM to decompose the MINLP problem into trivial sub-problems and solve it with the iterative approximation algorithm with low computation overhead. Numerical simulations with diverse emergency settings demonstrate that our framework outperformed baseline methods in terms of resource efficiency, service reliability and cost-effectiveness.

Acknowledgment

The authors thank Mitacs, Ciena, and ENCQOR for funding this research under the IT13947 grant.

References

[1] "Preface," in *5G NR: the Next Generation Wireless Access Technology*, E. Dahlman, S. Parkvall, and J. Sköld, Eds. Academic Press, 2018, pp. xv–xvi. [Online]. Available: https://www.sciencedirect.com/science/article/pii/B9780128143230000296

[2] [Online]. Available: https://www.o-ran.org/o-ran-resources

[3] S. Sahhaf, W. Tavernier, M. Rost, S. Schmid, D. Colle, M. Pickavet, and P. Demeester, "Network service chaining with optimized network function embedding supporting service decompositions," *Computer Networks*, vol. 93, pp. 492 – 505, 2015, cloud Networking and Communications II. [Online]. Available: http://www.sciencedirect.com/science/article/pii/S138912861500359X

[4] J. F. Riera, E. Escalona, J. Batallé, E. Grasa, and J. A. García-Espín, "Virtual network function scheduling: Concept and challenges," in *Smart Communications in Network Technologies (SaCoNeT), 2014 International Conference on*, June 2014, pp. 1–5.

[5] J. Ren, G. Yu, Y. He, and G. Y. Li, "Collaborative cloud and edge computing for latency minimization," *IEEE Transactions on Vehicular Technology*, vol. 68, no. 5, pp. 5031–5044, 2019.

[6] R. Gouareb, V. Friderikos, and A.-H. Aghvami, "Virtual network functions routing and placement for edge cloud latency minimization," *IEEE Journal on Selected Areas in Communications*, vol. 36, no. 10, pp. 2346–2357, 2018.

[7] O. Bushnaq, D. Mishra, E. Natalizio, and I. Akyildiz, *Unmanned aerial vehicles (UAVs) for disaster management*, 01 2022, pp. 159–188.

[8] F. Dolente, R. G. Garroppo, and M. Pagano, "A vulnerability assessment of open-source implementations of fifth-generation core network functions," *Future Internet*, vol. 16, no. 1, 2024. [Online]. Available: https://www.mdpi.com/1999-5903/16/1/1

[9] L. Bertizzolo, T. Tran, J. Buczek, B. Balasubramanian, R. Jana, Y. Zhou, and T. Melodia, "Streaming from the air : Enabling drone-sourced video streaming applications on 5g open-ran architectures," *IEEE Transactions on Mobile Computing*, vol. PP, pp. 1–1, 11 2021.

[10] M. L. Littman, "Markov games as a framework for multi-agent reinforcement learning," in *Proceedings of the Eleventh International Conference on International Conference on Machine Learning*, ser. ICML'94. San Francisco, CA, USA: Morgan Kaufmann Publishers Inc., 1994, p. 157–163.

[11] R. Lowe, Y. Wu, A. Tamar, J. Harb, P. Abbeel, and I. Mordatch, "Multi-agent actor-critic for mixed cooperative-competitive environments," 2020.

[12] M. Hong, T.-H. Chang, X. Wang, M. Razaviyayn, S. Ma, and Z.-Q. Luo, "A block successive upper bound minimization method of multipliers for linearly constrained convex optimization," *Mathematics of Operations Research*, vol. 45, 01 2014.

[13] M. Hong, M. Razaviyayn, Z.-Q. Luo, and J.-S. Pang, "A unified algorithmic framework for block-structured optimization involving big data: With applications in machine learning and signal processing," *IEEE Signal Processing Magazine*, vol. 33, no. 1, pp. 57–77, 2016.

[14] [Online]. Available: https://openairinterface.org/projects/

[15] N. Nikaein, M. K. Marina, S. Manickam, A. Dawson, R. Knopp, and C. Bonnet, "Openairinterface: A flexible platform for 5g research," *SIGCOMM Comput. Commun. Rev.*, vol. 44, no. 5, p. 33–38, Oct. 2014. [Online]. Available: https://doi.org/10.1145/2677046.2677053

[16] [Online]. Available: https://www.srslte.com/

[17] [Online]. Available: https://open5gs.org/open5gs/docs/

[18] D. Cruz, T. Cruz, V. Pereira, and P. Simões, "Designing a high-fidelity testbed for 5g-based industrial iot," in *Proceedings of the 22nd European Conference on Cyber Warfare and Security (ECCWS 2023), Athens, Greece (June 2023). DOI*, vol. 10, 2023.

[19] K. G. Panda, S. Das, D. Sen, and W. Arif, "Design and deployment of uav-aided post-disaster emergency network," *IEEE access*, vol. 7, pp. 102 985–102 999, 2019.

[20] Z. Yao, W. Cheng, W. Zhang, and H. Zhang, "Resource allocation for 5g-uav-based emergency wireless communications," *IEEE Journal on Selected Areas in Communications*, vol. 39, no. 11, pp. 3395–3410, 2021.

[21] I. Jawhar, N. Mohamed, J. Al-Jaroodi, D. P. Agrawal, and S. Zhang, "Communication and networking of uav-based systems: Classification and associated architectures," *Journal of Network and Computer Applications*, vol. 84, pp. 93–108, 2017.

[22] Y. Gao, J. Cao, P. Wang, J. Yin, M. He, M. Zhao, M. Peng, S. Hu, Y. Sun, J. Wang *et al.*, "Intelligent uav based flexible 5g emergency networks: Field trial and system level results," in *IEEE INFOCOM 2020-IEEE Conference on Computer Communications Workshops (INFOCOM WKSHPS)*. IEEE, 2020, pp. 138–143.

[23] A. Ranjan, B. Panigrahi, H. K. Rath, P. Misra, A. Simha, and H. Sahu, "A study on pathloss model for uav based urban disaster and emergency communication systems," in *2018 Twenty Fourth National Conference on Communications (NCC)*. IEEE, 2018, pp. 1–6.

[24] L. Chiaraviglio, L. Amorosi, F. Malandrino, C. F. Chiasserini, P. Dell'Olmo, and C. Casetti, "Optimal throughput management in uav-based networks during disasters," in *IEEE INFOCOM 2019-IEEE Conference on Computer Communications Workshops (INFOCOM WKSHPS)*. IEEE, 2019, pp. 307–312.

[25] M. Y. Arafat and S. Moh, "Localization and clustering based on swarm intelligence in uav networks for emergency communications," *IEEE Internet of Things Journal*, vol. 6, no. 5, pp. 8958–8976, 2019.

[26] R. Mao, B. Du, D. Sun, and N. Kong, "Optimizing a uav-based emergency medical service network for trauma injury patients," in *2019 IEEE 15th International Conference on Automation Science and Engineering (CASE)*. IEEE, 2019, pp. 721–726.

[27] C.-K. Wen, L.-S. Tsai, A. Shojaeifard, P.-K. Liao, K.-K. Wong, and C.-B. Chae, "Shaping a smarter electromagnetic landscape: Iab, ncr, and ris in 5g standard and future 6g," *IEEE Communications Standards Magazine*, vol. 8, no. 1, pp. 72–78, 2024.

[28] Q. Ye, W. Zhuang, X. Li, and J. Rao, "End-to-end delay modeling for embedded vnf chains in 5g core networks," *IEEE Internet of Things Journal*, vol. 6, no. 1, pp. 692–704, 2019.

[29] D. P. Bertsekas and R. G. Gallager, *Data networks*, vol. 2.

[30] C. Pham, K. K. Nguyen, M. Cheriet, and A. Shabah, "Joint horizontal and vertical backup for highly reliable telemedicine services," in *ICC 2023 - IEEE International Conference on Communications*, 2023, pp. 4428–4434.

[31] H. Lyu, "Convergence and complexity of block coordinate descent with diminishing radius for nonconvex optimization," 2021.

[32] A. Chiu, G. Choudhury, G. Clapp, R. Doverspike, M. Feuer, J. Gannett, J. Jackel, G. Kim, J. Klincewicz, T. Kwon, G. Li, P. Magill, J. Simmons, R. Skoog, J. Strand, A. Lehmen, B. Wilson, S. Woodward, and D. Xu, "Architectures and protocols for capacity efficient, highly dynamic and highly resilient core networks [invited]," *Journal of Optical Communications and Networking*, vol. 4, pp. 1–14, 01 2012.

[33] Coronet: Testbeds, demonstration, and lessons learned. [Online]. Available: https://opg.optica.org/jocn/abstract.cfm?uri=jocn-7-3-A447

[34] V. Garg, *Wireless Communications & Networking*, 1st ed. San Francisco, CA, USA: Morgan Kaufmann Publishers Inc., 2007.

[35] S. Dräxler and H. Karl, "Spring: Scaling, placement, and routing of heterogeneous services with flexible structures," in *2019 IEEE Conference on Network Softwarization (NetSoft)*, 2019, pp. 115–123.

[36] S. Schneider, N. P. Satheeschandran, M. Peuster, and H. Karl, "Machine learning for dynamic resource allocation in network function virtualization," in *2020 6th IEEE Conference on Network Softwarization (NetSoft)*, 2020, pp. 122–130.

[37] A. Boskovic, Q. Chen, D. Kufel, and Z. Zhou, "Online learning and matching for resource allocation problems," 2019. [Online]. Available: https://arxiv.org/abs/1911.07409

PLACE PHOTO HERE

Chuan Pham He received the BS degree from Hochiminh City University of Transport, master degree from HoChiMinh City University of Sciences and Ph.D degree from Kyung Hee University, in electrical and computer engineering, in 2004, 2008 and 2017, respectively. He was a postdoc fellow in Department of Computer Science and Engineering, Kyung Hee University from August, 2017. Since 2018, he has been a postdoc fellow in Synchromedia - École de Technologie Supérieure, Université du Québec.

PLACE PHOTO HERE

Kim. Nguyen is Associate Professor in the Department of Electrical Engineering at the University of Quebec's Ecole de technologie supérieure, Montreal, Canada. He has a Ph.D. from Concordia University in Electrical and Computer Engineering. In the past, he served as CTO of Inocybe Technologies (now is Kontron Canada), a leading company in software-defined networking (SDN) solutions. He was the architect of the Canarie's GreenStar Network and also involved in establishing CSA/IEEE standards for green ICT. He has led R&D in large-scale projects with Ericsson, Ciena, Telus, and InterDigital.